

# A Queryable Model Store Architecture for Dynamic Multi-Layer Algebra

Gergely Gembela

*Department of Automation and Applied Informatics*

*Budapest University of Technology and Economics*

gergelygembela@edu.bme.hu

## **Abstract.**

The framework powering the Dynamic Multi-Layer Algebra project was initially designed to process the source code of models, similarly to how traditional compilers process code. It processed text-based source code, and after a set of transformations, the framework validated, analysed, or transformed models through multiple separate executions that involved parsing and processing the model many times. As we explored real-world use cases and designed development tooling, we decided that the framework would benefit greatly from an architectural overhaul. The new architecture is built around the idea of DMLA models being a dynamic, queryable, long-lived source of information. The new framework is centered around a metamodel database (the SymbolDB) that stores model state, metadata (such as the origin of elements), detailed descriptions, and traces for both code-related and modeling-space errors. The SymbolDB exposes model elements with rich contextual information and metadata, supports direct manipulation of model elements along with dynamic code execution, and was built with incrementality in mind. Our new design enables entirely new use cases and simplifies the creation of new tooling, such as Compiler as a Service (CaaS) features or model visualization. This paper provides an overview of the new architecture and introduces new tools built around the new execution engine.

**Keywords:** Multi-Level Modeling; Metamodeling; DMLA; Systems Engineering, Dynamic Multi-Layer Algebra

## 1 Introduction

The software stack underlying the Dynamic Multi-Layer Algebra project has been under development for more than a decade. The initial concept and architecture were designed around single-shot executions and a traditional compiler pipeline. As the language evolved, new requirements for developer tooling (e.g., syntax highlighting, recommendations, real-time error checking) had to be introduced, and new use cases emerged that pushed the traditional architecture to its limits. Throughout the past two years, the entire software stack has undergone a major architectural overhaul, reinterpreting the role of models. In the new architecture, models are long-lived and queryable. There are multiple paths for manipulation and interaction with models, opening up the modeling space for various new tools (e.g., the new model visualization engine), and allowing for complex model analysis and transformation from external components.

Earlier, we proposed a new (compiler) frontend architecture for DMLA [1]. The new frontend adopts the Compiler as a Service (CaaS) paradigm, exposing rich source-code related metadata through the Language Server Protocol. To simplify the implementation of CaaS and analysis features, metadata is stored alongside modeled symbols. The new compiler brings about a shift in the model validation paradigm: the same way as CaaS opened up compilers, the new architecture provides interfaces exposing model elements and model state. In this paper, we introduce the redesigned architecture of the modeling environment, created around the new compiler frontend. The redesign eliminated a significant technical debt and shifted the purpose and lifecycle of the DMLA environment, enabling entirely new practical solutions built on top of DMLA. This paper presents the new architecture, APIs, and tools we created to make DMLA more practical to adopt and adapt, and shows new use cases we kept in mind while designing the new toolset.

Section 2 provides background, shows widely used solutions, and introduces other solutions that influenced our design. Section 3 briefly summarizes the relevant background related to DMLA itself, and Section 4 introduces the new architecture. Finally, Section 5 concludes the paper and outlines future work.

## 2 Background and Related Work

Traditionally, compilers were thought of as complex black-box-like programs that convert text to machine code. With the introduction of the Compiler as a Service (CaaS) paradigm, compilers have become APIs that expose meta-models of the source code they process. One of the first, and arguably the most exemplary CaaS project is Roslyn<sup>1</sup> [2], the compiler platform of the .NET framework. Roslyn opens up the inner workings of the .NET compiler platform, provides various points of extensibility, and publishes the internal representation of the source code it processes at multiple stages. Most modern compilers, such as Kotlin’s K2 or Rustc, followed suit [3], as CaaS features and more open pipelines have the potential to eliminate the need for maintaining parallel implementations of tools capable of processing a language; only one codebase needs to *understand* the language. For this reason, more open compilers are universally beneficial for both compiler developers and users.

Some modern compilers, such as *rustc*<sup>2</sup> (the compiler of the Rust language), are adopting a query-based demand-driven approach<sup>3</sup> for compilation, in which the results of queries are cached, and incrementality is achieved by reusing cached results. Instead of code being passed down a predefined pipeline of compilation passes, queries result in a chain of subsequent queries until the desired result (e.g., the type of a certain parameter, or the source range of the definition of a symbol with a certain identifier) is reached. This architecture has

---

<sup>1</sup><https://learn.microsoft.com/en-us/visualstudio/extensibility/dotnet-compiler-platform-roslyn-extensibility?view=visualstudio>

<sup>2</sup><https://rustc-dev-guide.rust-lang.org/>

<sup>3</sup><https://ollef.github.io/blog/posts/query-based-compilers.html>

many advantages for compilers with the ultimate goal of transforming source code to a binary, but mostly needs to serve partial information of the source code to provide quick editor support. While we have considered a demand-driven architecture for DMLA, the benefits of a query-based approach are not as apparent as they are for compilers of general-purpose programming languages, as our environment relies on always having a complete and up-to-date picture of the modeling space.

### 3 DMLA

DMLA is a multi-level modeling framework and environment that operates primarily on text-based (static) and dynamically created model elements. Multi-level modeling is a direction of metamodeling that combines types with their instantiations (classes and objects in object-oriented programming (OOP)) and allows an arbitrary number of instantiations, thereby reducing accidental complexity resulting from the structure of regular models that enforce this separation. In DMLA, the introduction of new types (and instances) is referred to as *refinements* [4]. DMLA models are composed of entities (elements that are similar to classes in OOP), slots (analogous to properties in OOP), and operations. Entities, operations, and slots can be refined, and annotations can be applied to all elements to add metadata or constraints. Annotations include, for example, contract, omission, type, cardinality, and final constraints. Model elements are organized in packages, and packages may reference other packages when loaded in a common modeling space.

#### 3.1 The Bootstrap

The DMLA runtime is an environment designed to accommodate DMLA models. It provides the background for managing the modeling space, but the bootstrap is the component that actually defines how DMLA models work. The bootstrap has control over model validation and provides a modeled layer through which domain models can interact with the environment beyond the modeling space. A bootstrap is generally a package holding a certain set of required model elements that form a self-describing partial specification of the modeling environment itself [5]. While model spaces cannot exist without a *Bootstrap* package, the environment was designed around the idea that the bootstrap is customizable. While we aim to support maximal customizability, the rules that the bootstrap implements and around which the runtime was designed must stay consistent. For a package to be a valid *Bootstrap*, it must fulfill a set of requirements. First and foremost, every bootstrap element has to be in the *Bootstrap* package, and a list of model elements such as the *Base* entity, several structural primitives such as the *Base.Fields* slot, and several operations are expected to be defined. As it would be largely impractical to model all DMLA functionality, the environment exposes so-called built-in operations. Built-ins implement low-level functionality such as mathematical operations and model

manipulation utilities, and are only accessible from the bootstrap. The bootstrap may either expose or hide these operations, but they cannot be called directly from other packages. Bootstraps can expect a set of built-ins to be available at all times. Built-ins include operations for dynamic instantiation, querying metadata, interfacing with external code, and mathematical or other general-purpose operations that may benefit from static (i.e., not interpreted) implementations.

### 3.2 The D# Language

D# (DSharp) is a text-based custom language for describing DMLA models. It is the primary tool for creating DMLA models containing both structural and behavioral model elements. The syntax of the language was inspired by C#, and it aims to be similar enough to most modern languages that it is easy to read and understand. Due to the dynamic nature of the DMLA model space, D# files should be viewed as snapshots of a model’s state. We intend for the language to be usable for exactly that: a persistent snapshot of the model space.

D# source files are organized in *workspaces*. Workspaces contain domain models organized under a common root directory. When the DMLA environment is started in a specific workspace, a workspace session is started. During its initialization, all models in the workspace are loaded into a shared modeling space. When DMLA receives a query to execute a specific operation of the modeling space, an execution session is started. The runtime API, accessible both through modeled operations and external technologies, is capable of creating two kinds of model elements: *dynamic* and *static*. Dynamic model elements are not added to the workspace session’s modeling space; they only exist in the extended modeling space linked to the execution session in which they were created, and are disposed of when the execution session terminates. Conversely, static entities created during runtime are added to the modeling space and are persisted beyond the lifecycle of execution sessions. Each workspace requires exactly one *Bootstrap* model to be loaded. In most cases, the *default Bootstrap* linked by the launcher automatically is sufficient, but in development scenarios, the Bootstrap may also be supplied as part of the workspace itself. Domain models sharing a workspace can reference and access elements across packages. With the new architecture, each workspace may define a domain package called *workspace*, in which model elements can be used to store workspace-specific configuration, as the new runtime can access model elements and modeled values early on in the pipeline.

## 4 The DMLA Model Ecosystem

The new DMLA stack provides an extended set of CaaS features and transforms DMLA from a traditional compiler to a service that stores and operates on multi-level models. We took the CaaS pattern as a starting point for designing a set of APIs for interacting with models on a deeper level, which goes beyond editor

support. In the new architecture tooling, utilities, and runtime functions were split into distinct components and are developed and versioned independently. Fig. 1 shows an abstract overview of the new DMLA stack.

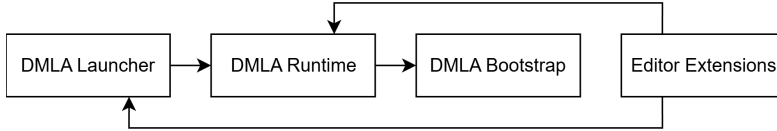


Figure 1: The components of the new DMLA stack and their dependencies

The new stack is made up of 4 distinct components: The DMLA Launcher, DMLA Runtime, DMLA Bootstrap, and editor support extensions (currently a VS Code Extension). While the components depend on each other (the arrows in Fig. 1 depict dependencies), they are developed under distinct projects and use completely different underlying technologies.

## 4.1 The Launcher

The DMLA launcher was introduced to automate the installation and maintenance of dependencies and components and simplify launching the runtime. Previously, it was required to separately install and configure the required dependencies, the appropriate runtime version, the VS Code extension, and Bootstrap. In addition, dependency- and runtime-specific launch parameters were needed, which often resulted in an unstable CLI. The new launcher eliminates this complexity by providing a concise, stable command-line interface for managing and using all DMLA components, independent of dependency or runtime versions. The launcher can install and update all components and dependencies, thus only a single download is required to get started with DMLA. Once the launcher is installed, it checks for updates in the background on a daily basis and prompts the user to update when the workspace session is terminated.

## 4.2 The Runtime

The runtime is the main component in the new architecture, as it contains the entire compiler framework, including the lexer, parser, and transformation APIs, as well as the SymbolDB – a central database that hosts and maintains model state.

Fig. 2 provides an abstract overview of the runtime’s internal architecture. The first row of components forms the compiler frontend. The frontend is responsible for transforming source code into symbols. The SymbolDB (the DMLA middleend) is the central component of the entire system. It keeps track of all model elements along with their relations, state, and related metadata, such as their position in the model source (of static model elements originating from D# code). The interpreter operates on a projection of the SymbolDB, and even dynamic model changes, such as new instantiations or changes of slot values

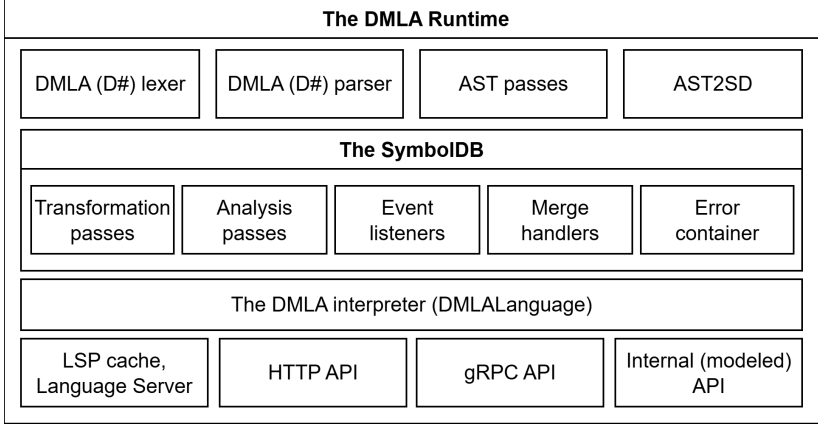


Figure 2: The internal components of the DMLA runtime

are committed to the SymbolDB. The SymbolDB stores behavioral elements in an intermediate representation that is ready to be executed right away. However, execution only starts when DMLA is invoked from the command line interface or when the execution of a specific operation is requested through the API. The last row of components facilitates internal and external interaction with the modeling space through various APIs.

In previous work [1], we redesigned the DMLA parser from the ground up and opened up the compilation pipeline at various stages. The first row of components in Fig. 2 is the frontend of the D# compiler; the components are listed in the order they process code. Both the lexer and parser use our custom logic. The lexer turns the code into a token stream, while the parser builds separate ASTs for modeled packages. After their construction, ASTs are exposed for transformation and analysis through multiple AST passes, such as transformations that flatten deep trees built to avoid left recursion or the resolution of error recoveries. The last step of the frontend (*AST2SD*) transforms the syntax trees of source files – and incremental changes to them – into symbols. The new parser is composed of extensible steps, with each stage giving us full control over the processed code at every step of the compiler frontend as text is transformed into tokens and syntax trees are built. As a result, adding new language features has become much quicker, and implementing convenience features (such as implicit meta specifications or preliminary type-checking) usually boils down to implementing new SymbolDB or AST transformation passes, giving us great flexibility when designing language extensions or new elements. The control we have over even low-level components has helped us create better error messages, provide language-specific recovery strategies for processing error-containing code, and build rich, cacheable internal representations for each stage to work with.

The following subsections provide a deep dive into the key components of the new runtime.

### 4.2.1 The SymbolDB

The SymbolDB is a central database that stores *Symbols*. In DMLA, there are five types of *Symbols*: Packages, Entities, Annotations, Operations, and Slots. Every symbol (even dynamically created ones) has an origin that holds information on how the element was created. For elements originating in textual source code, intermediate formats such as the corresponding tokens and abstract syntax trees (or subtrees) are also stored, so that incremental changes can use cached subtrees, and editor features have the most information possible to work with. Details such as comments or line endings (often referred to as trivia in CaaS terms) are also stored and used for in-editor documentation purposes.

The D# language was designed to be practical and easy to use, and even though it carries complete models, reconstructing them requires several steps referred to as transformation passes. When the parser processes source files, it extracts the subset of relations the source code carries explicitly. At this point, a SymbolDB instance contains several missing implicit details, and transformation passes need to be applied in order to reach a fully complete state. Analyzer passes are also executed during model processing to ensure that most errors are discovered before the interpreter starts executing modeled code. Once all passes are completed (i.e., there are no pending transformations), the SymbolDB enters a state we call *fully processed*. In this state, the SymbolDB can be queried and provides access to all information the model space holds. Once a SymbolDB is fully processed, there are two ways it can change: the CaaS integration may report a change that triggers the reprocessing of affected elements, or elements are dynamically changed through a modeled or external API.

Every change, including updates to the model space via static (CaaS) or dynamic (modeled or API-based) methods, results in a state change in the SymbolDB. The SymbolDB publishes various events that other components can react to. Events are fired on every state change that may be relevant to other components, such as the completion of each transformation pass, when instances are merged, or when updates are received.

When the parser processes incremental changes, it only reparses the packages affected by the change. For each update, the parser derives a partition of the actual SymbolDB that contains all unchanged dependencies of the updated packages, and processes the new version of each updated package. The parser only needs to build ASTs for symbols that are affected by the change, as ASTs are cached for all files. When the changed module is parsed, all SymbolDB passes are executed so that the partition reaches a fully processed state. If the new SymbolDB partition is successfully processed (i.e., every pass completes successfully and no error is generated that prevents merging the partition), it is merged back to the original SymbolDB, and the appropriate merge and update events are fired.

With the previous architecture, dynamically created model elements were only reflected and accessible from within the model itself. In contrast, these elements are stored in the SymbolDB and are accessible through all APIs.

### 4.2.2 Error Handling

The runtime handles errors in two main ways. Errors discovered during transformation and analysis passes are collected in the SymbolDB’s error container, while dynamic errors (errors encountered during the execution of modeled behavior, or generated by invalid dynamic interactions with the modeling API) result in exceptions. The parser and all passes attempt to recover from errors so that the maximal number of errors is caught in a single parsing and processing attempt. Errors caught during parsing and transformations have severities (of *info*, *warning*, or *error*) and are reported through the logging system and the language server. Besides error messages, all errors contain metadata that pin-points potential symbols or AST nodes related to the cause of the error. Besides severity, two other mandatory indicators help the system determine the way errors are treated: errors may either block the merging of SymbolDB instances or prevent the runtime from executing elements of SymbolDB instances that contain such errors. All frontend and middleend errors have identifiers that identify their stage, error group, and specific type. The *workspace* domain package can be used to suppress errors of specific kinds using their identifiers.

Depending on the severity and complexity of runtime exceptions, they are either handled in modeled code or by the runtime. We are working on transferring as many runtime exceptions into modeled exceptions as possible so that the modeled code can handle all non-critical errors.

### 4.3 Interaction with Models

The runtime provides various methods for interacting with models and the modeling environment. As shown in Fig. 3, model elements are exposed through multiple APIs over different protocols. Editor support is primarily provided through the Language Server Protocol (LSP). The LSP API provides editor-related access to the model state, and additional editor-specific metadata, such as implicit metaslots and metaentities, documentation comments, and recommendations, are computed on demand when requested by the language client as described in Section 4.3.2.

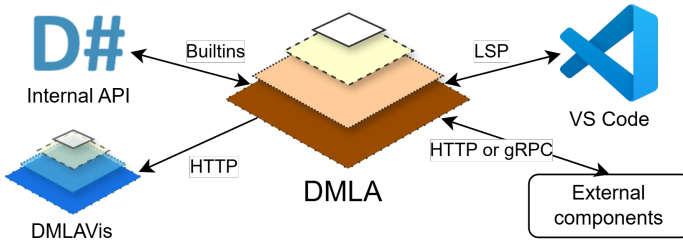


Figure 3: Model manipulation channels

More generic modeling APIs are accessible over both gRPC and HTTP protocols, providing real-time access to the SymbolDB and exposing endpoints for



operation invocations. The ultimate goal of the generic APIs is to simplify the integration of DMLA models into other systems and provide a flexible platform for bi-directional model transformations. DMLA is not an open source project, and for now, the protocol buffer-based IDL specification and the automatically generated OpenAPI specification of the gRPC and HTTP APIs are only available internally, as they are subject to major changes while we are working towards the integration of different tools and external model transformations.

#### 4.3.1 DMLAVis

DMLAVis is an entirely new project in the stack. Built on top of the SymbolDB, the DMLAVis project provides model visualization for DMLA models. The official visual syntax is still under development, but model visualization has already proven to be a useful addition to our official set of tools. The DMLA visualization engine uses the HTTP API to access the SymbolDB, and for now, it has read-only access to the modeling space. Model elements are connected and laid out automatically, but to make the visualization more flexible, the elements can be manually reorganized, and details can be hidden (though changes are local to the metamodel used by the visualization engine). Fig. 4 shows a partial visualization of a DMLA domain model.

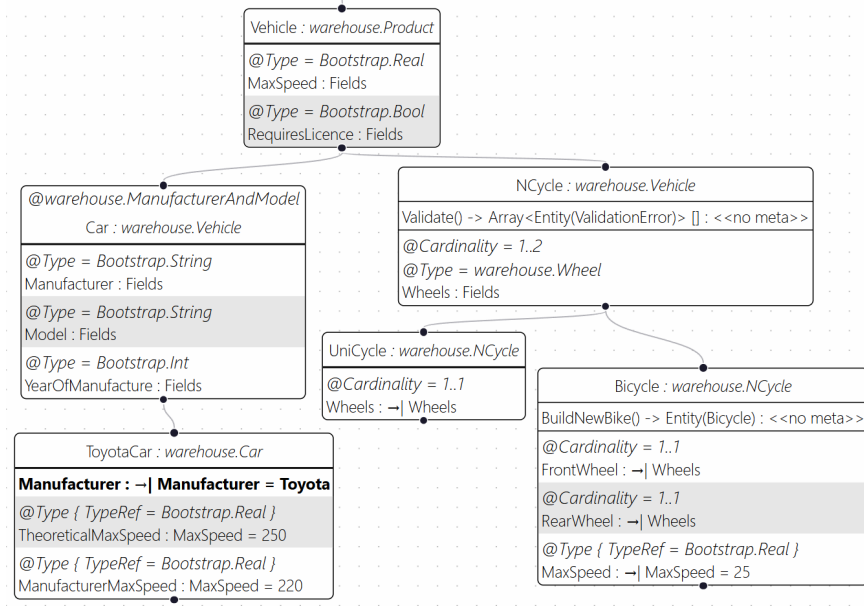


Figure 4: A DMLAVis example (showing a part of the warehouse domain)

### 4.3.2 The Language Server

When it comes to CaaS features, the Language Server is the most important part of the runtime. The Language Server adheres to the official Language Server Protocol specification [6]. Previously, we had to traverse the ASTs multiple times to extract information required by editor features. In the new runtime, symbols and their metadata stored in SymbolDB drastically simplify the implementation of language server features, as features such as inlay hints, recommendations, and autocomplete essentially become queries.

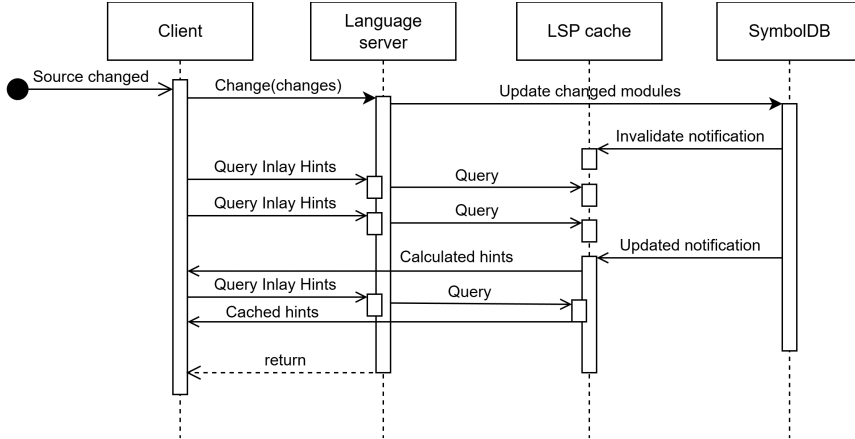


Figure 5: The LSP cache in action

The language server protocol was designed to support asynchronous communication between the server (in our case, the DMLA runtime) and the client (VS Code). For our language server (LS) to benefit from this, we created a custom LSP cache, a system for decoupling LSP queries from parsing while caching results for each request type and context. The cache is updated using events published by the SymbolDB, allowing the LS to compute the results for complex queries only once and serve all requests as quickly as possible. The event-driven nature of the LSP cache enables events fired by SymbolDB updates or stages to efficiently manage cache state, as queries can be queued until the compiler pipeline reaches the exact stage at which the results can be recomputed. Fig. 5 shows an abstract representation of how the cache handles inlay hint requests<sup>4</sup>. Inlay hints are requested by the editor multiple times even between changes, and are usually requested while the pipeline is still processing changes. When a change occurs, the SymbolDB invalidates the LSP cache of the packages affected by the change, and requests are queued until the SymbolDB pass required for inlay hints completes successfully, or a successful merge occurs. Upon notification, inlay hints are calculated and cached, and until a change invalidates the LSP cache, all requests are served from cache. For less frequently requested cache items, the cache delays computing the result until the first request is received.

<sup>4</sup><https://www.jetbrains.com/help/idea/inlay-hints.html>

## 4.4 Integration and Automation

For each of the new components, CI/CD pipelines were set up, automating builds, testing, and the creation of new releases. The DMLA stack is tested in many different ways. We employ multiple testing methods targeting different components of the pipeline. The launcher, the frontend, the middleend, and the interpreter are tested using separate unit tests, and modeled tests implemented in D#. These tests aim to verify the correctness of the entire stack, including the default Bootstrap and D# language features.

## 4.5 DMLA Sandboxes

DMLA *sandboxes* contain specific snapshots of each component (with updates disabled) that reflect the state of the framework at the time of writing. Sandboxes are based on development containers<sup>5</sup>, and require minimal configuration and no installation on the user’s side, as every component is preinstalled in the container image, and the workspace is pre-configured to be usable out of the box. This is particularly valuable, as it allows distributing a ready-to-use environment tailored to a specific paper or domain. DMLA sandboxes can also be installed on GitHub Codespaces<sup>6</sup> or any other service that supports dev-containers.

## 5 Conclusion and Future Work

The new architecture and tooling powering the DMLA modeling environment bring the environment up to modern standards. With the enhancements, the capabilities of DMLA are expanding, and we are already working on use cases that depend heavily on long-lived queryable DMLA models. We are already leveraging the benefits of the new architecture, as most new language features only require changes or additions at points naturally designed to be extensible. One of the most notable improvements is that we can implement LSP features within minutes to tens of minutes rather than hours or days. We can iteratively improve language features with ease, as new language features such as modeling-time type checking or shorthands for common annotations are gradually introduced as SymbolDB transformation passes.

While the entire stack was built with incrementality in mind, the current module-level incremental updates are not detailed enough for our next integral goal: incremental validation. Structures and utilities are included in both the SymbolDB and its underlying infrastructure, but we have yet to achieve fine-grained (entity or even slot-level) incremental validation. By extending the event system already present in the runtime, incremental validation would accelerate transformation-based use cases by drastically reducing the number of transformations required, as events could be fired for the minimal set of (co-)dependent elements affected by the changes.

---

<sup>5</sup><https://containers.dev/>

<sup>6</sup><https://github.com/features/codespaces>

Fully processed SymbolDBs were designed to have a corresponding textual (D#) representation/snapshot. We plan to implement reverse AST transformations for dynamically created model elements. This transformation will allow for the trivial persistence of dynamically created non-dynamic entities and enable the easy implementation of default formatting.

While the launcher can already start multiple components (e.g., the LS and the APIs) in the same workspace session, for now, all changes made to a modeling space are persistent for the workspace session and are lost when the session is terminated. The launcher will have to be made workspace-session aware. With integrated session management, it could convert command-line execution requests to API calls and redirect them to the appropriate runtime instance managing the workspace in which the command was issued.

With the SymbolDB exposing various capabilities of our models, in the current climate, it is a natural next step to extend the DMLA stack with a component that adapts the SymbolDB to the Model Context Protocol<sup>7</sup> so that AI agents can manipulate models. With the runtime exposing multiple representations, this integration will need to take into account the modalities and representations that AI models could process or generate the best.

## Acknowledgment

The author would like to express his thanks to Gergely Mezei for his support as a scientific advisor.

## References

- [1] G. Gembela and G. Mezei, “A modern compiler frontend for dynamic metamodeling,” in *Proceedings of the Automation and Applied Computer Science Workshop 2025*, pp. 242–247, 2025.
- [2] M. Vasani, *Roslyn Cookbook*. Packt Publishing, 2017.
- [3] G. Gembela and G. Mezei, “Extensibility in modern compiler platforms,” in *Proceedings of the Automation and Applied Computer Science Workshop 2024 : AACS’24*, pp. 278–283, 2024.
- [4] M. Jeusfeld, G. Mezei, and S. Bácsi, “Deeptelos and DMLA – A contribution to the MULTI 2022 collaborative comparison challenge,” in *Proceedings of the ACM/IEEE 25th International Conference on MODELS: Companion Proceedings*, (New York, NY, USA), ACM, 2022.
- [5] G. Mezei, D. Urban, S. Bacsí, F. A. Somogyi, D. G. Palatinszky, and Z. Theisz, “A bootstrap for self-describing, self-validating multi-layer metamodeling,” in *Proceedings of the Automation and Applied Computer Science Workshop 2019*, pp. 28–38, 2019.
- [6] *The Language Server Protocol*. <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>.

---

<sup>7</sup><https://modelcontextprotocol.io/>